

REPASO INTEGRADOR · PARTE 3 DE 3

Programación Orientada a Objetos en C++

Repaso Pre-Parcial

Caso de estudio: Sistema de Biblioteca — Módulos 8 a 14 y Cierre

```
class ObjetoDeEstudio {  
private:  
    Conocimiento conocimiento;  
public:  
    void prepararParcial();  
};
```

Cadenas tipo C · Recursividad · Templates · Buenas Prácticas · Errores Frecuentes · Ejercicio Integrador · Simulacro de Parcial

Programación Orientada a Objetos - Leonardo Brambilla - FCyT - UADER

MÓDULO

8

≈ 12 minutos

Cadenas tipo C

Objetivos de esta sección

- Manipular cadenas `char[]` con las funciones de `<cstring>`.
- Comparar `char[]` con `std::string` y justificar cuándo usar cada una.

char[], funciones de <cstring> y std::string

- ▶ char[] es un arreglo de caracteres terminado en '\0' (carácter nulo); no crece automáticamente.
 - strlen(s): longitud sin contar '\0'.
 - strcpy(dst,src): copia.
 - strcmp(a,b): compara (0 si son iguales).
 - strcat(dst,src): concatena.
- ▶ std::string gestiona su propia memoria dinámicamente, crece sola y soporta operadores (+, ==, <=).
- ▶ Sistemas legados u orientados a bajo nivel (Oracle Forms, C embebido) todavía exigen conocer char[].

char[] vs std::string

Operación	char[]	std::string
Longitud	strlen(s)	s.length()
Copiar	strcpy(dst,src)	dst = src
Comparar	strcmp(a,b)==0	a == b
Concatenar	strcat(dst,src)	dst += src
Gestión memoria	manual	automática

 **Error frecuente:** Usar '==' para comparar char[] (compara direcciones de memoria, no contenido); debe usarse strcmp.

? ¿Qué imprime este fragmento? char a[]="biblioteca"; char b[]="biblioteca"; cout << (a==b);

Resumen — Cadenas tipo C

✓ Ideas clave

- `char[]` requiere funciones de `<cstring>`: `strlen`, `strcpy`, `strcmp`, `strcat`.
- `std::string` es más segura y expresiva; se prefiere salvo en contextos de bajo nivel.

⚠ Errores frecuentes

- Comparar `char[]` con `==` en vez de `strcmp`.
- Desbordar el arreglo destino en `strcpy/strcat` (buffer overflow) por falta de espacio reservado.

🔑 Ejercicio rápido

Escribí una función `'bool mismoTitulo(char* a, char* b)'` usando `strcmp`, y su equivalente usando `std::string`.

MÓDULO

9

≈ 20 minutos

Recursividad

Objetivos de esta sección

- Identificar caso base y caso recursivo en un problema.
- Resolver ejercicios de recursividad (factorial, Fibonacci, recorridos).
- Comprender la pila de llamadas y cómo probar una función recursiva.

Caso base, caso recursivo y pila de llamadas

- ▶ **Caso base:** condición que detiene la recursión sin volver a llamarse a sí misma.
- ▶ **Caso recursivo:** la función se llama a sí misma con una versión más pequeña del problema, acercándose al caso base.
- ▶ Cada llamada recursiva agrega un marco (stack frame) a la pila de llamadas; sin caso base se produce stack overflow.
- ▶ Ejemplo clásico: $\text{factorial}(n) = n * \text{factorial}(n-1)$, con $\text{factorial}(0) = 1$ como caso base.

factorial.cpp

```
int factorial(int n) {  
    if (n == 0) return 1;      // caso base  
    return n * factorial(n-1); // caso recursivo  
}  
  
factorial(3)  
= 3 * factorial(2)  
= 3 * (2 * factorial(1))  
= 3 * (2 * (1 * factorial(0)))  
= 3 * (2 * (1 * 1)) = 6
```

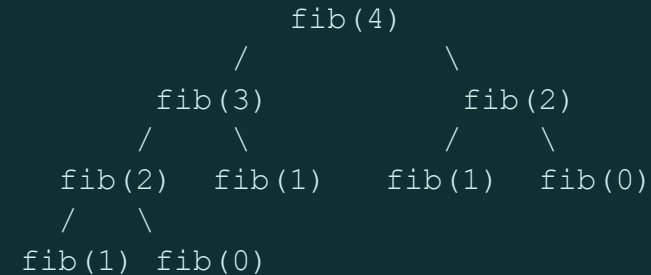
 **Error frecuente:** Omitir el caso base o no acercarse a él en cada llamada, provocando recursión infinita y stack overflow.

? ¿Cuántos 'marcos' quedan apilados en la pila de llamadas justo antes de que `factorial(0)` retorne?

Fibonacci, árbol de llamadas y cómo probar recursión

- ▶ $\text{Fibonacci}(n) = \text{Fibonacci}(n-1) + \text{Fibonacci}(n-2)$, con dos casos base: $\text{Fibonacci}(0)=0$, $\text{Fibonacci}(1)=1$.
- ▶ A diferencia del factorial, cada llamada genera DOS llamadas recursivas: se forma un árbol, no una cadena lineal.
- ▶ Para probar una función recursiva: verificar el caso base directamente, luego un caso pequeño a mano, y luego confiar en la inducción para casos mayores.
- ▶ Aplica también a recorridos recursivos de estructuras (por ejemplo, recorrer recursivamente las categorías de un catálogo de Biblioteca).

Árbol de llamadas — fib(4)



Casos base: $\text{fib}(0)=0$, $\text{fib}(1)=1$

 **Error frecuente:** Confundir el orden de evaluación asumiendo que $\text{fib}(n-1)$ y $\text{fib}(n-2)$ se calculan "al mismo tiempo"; en realidad se resuelven secuencialmente en la pila.

? ¿Por qué esta versión de Fibonacci es ineficiente para valores grandes de n ? (pista: conta cuántas veces se recalcula $\text{fib}(1)$).

Resumen — Recursividad

✓ Ideas clave

- Toda función recursiva necesita caso base y caso recursivo que se acerque a él.
- Cada llamada ocupa un marco en la pila; sin caso base hay stack overflow.
- Fibonacci genera un árbol de llamadas (dos ramas), a diferencia de la cadena lineal del factorial.

⚠ Errores frecuentes

- Olvidar o mal ubicar el caso base.
- Caso recursivo que no reduce el problema (no converge al caso base).
- Confundir el orden real de ejecución en la pila de llamadas.

📝 Ejercicio rápido

Escribí una función recursiva 'int contarLibros(Nodo* n)' que recorra una lista enlazada de libros y cuente sus elementos, indicando caso base y caso recursivo.

MÓDULO

10

≈ 18 minutos

Templates

Objetivos de esta sección

- Implementar Function Templates.
- Implementar Class Templates.
- Comparar ambos y relacionarlos con `vector<T>`, `stack<T>` y `queue<T>`.

Function Templates

- ▶ Programación genérica: escribir código que funcione para múltiples tipos sin duplicarlo.
- ▶ `template<typename T>` antes de una función la convierte en genérica; T se reemplaza por el tipo real en cada uso.
- ▶ El compilador instancia automáticamente una versión concreta de la función para cada tipo con el que se la invoca.
- ▶ Ejemplos típicos: `mayor<T>()` para comparar dos valores, `intercambiar<T>()` para intercambiarlos.

templates.cpp

```
template <typename T>
T mayor(T a, T b) {
    return (a > b) ? a : b;
}

template <typename T>
void intercambiar(T& a, T& b) {
    T temp = a; a = b; b = temp;
}

cout << mayor(3, 7);           // int
cout << mayor(2.5, 1.1);       // double
cout << mayor<string>("a","b"); // string
```

? ¿Podría usarse `mayor<T>()` para comparar dos objetos `Libro`? ¿Qué requisito debería cumplir la clase `Libro`?

Class Templates

- ▶ Una Class Template genera una clase completa parametrizada por tipo, no sólo una función.
- ▶ Se instancia indicando el tipo entre '<>' al declarar el objeto: Caja<int>, Caja<Libro>.
- ▶ vector<T>, stack<T> y queue<T> de la STL son Class Templates que ya usamos sin haberlas escrito nosotros mismos.
- ▶ Permite, por ejemplo, un Caja<Libro> o un Caja<Socio> reutilizando exactamente el mismo código.

Caja.h

```
template <typename T>
class Caja {
private:
    T contenido;
public:
    Caja(T c) : contenido(c) {}
    T getContenido() const {
        return contenido;
    }
};

Caja<Libro> c1(elQuijote);
Caja<int> c2(42);
```

 **Error frecuente:** Olvidar volver a escribir 'template<typename T>' antes de CADA método definido fuera de la clase.

 ¿Cuál es la diferencia clave entre Function Template y Class Template en cuanto a lo que generan?
Function Template genera funciones concretas; Class Template genera clases completas concretas.

Function Template vs. Class Template

- ▶ Ambos evitan duplicar código para distintos tipos, aplicando el mismo principio de programación genérica.
- ▶ La STL (vector, stack, queue) es el mejor ejemplo cotidiano de Class Templates que serán usados en el curso.

Comparación

Aspecto	Function Template	Class Template
Genera	una función concreta	una clase concreta
Sintaxis de uso	mayor(3,7)	Caja<int> c(3)
Ejemplo STL	std::max<T>()	vector<T>, stack<T>, queue<T>
Se instancia con	inferencia de tipo	T explícito entre < >

? ¿Por qué `stack<Libro>` y `queue<Libro>` son útiles para modelar, respectivamente, una pila de devoluciones y una cola de reservas en la Biblioteca?

Resumen — Templates

✓ Ideas clave

- Los templates permiten programación genérica: un solo código para múltiples tipos.
- Function Template genera funciones;
- Class Template genera clases completas.
- `vector<T>`, `stack<T>` y `queue<T>` son Class Templates de la STL que usaremos más adelante.

⚠ Errores frecuentes

- Olvidar repetir `'template<typename T>'` en cada método definido fuera de la clase.
- Instanciar una Class Template sin indicar el tipo entre `'< >'`.
- Usar una función template con un tipo que no soporta los operadores requeridos (por ejemplo, `mayor<T>()` sin `operator>` definido).
- Regla práctica: los templates normalmente se implementan en el archivo `.h`, porque el compilador necesita ver su implementación para generar el código al instanciarlos.

🔑 Ejercicio rápido

Escribí una Function Template `'T sumar(T a, T b)'` y probala con `int`, `double` y con dos objetos Complejo del Módulo 5 (requiere `operator+`).

Buenas prácticas de diseño orientado a objetos

- ▶ Responsabilidad única: cada clase debe tener un solo motivo para cambiar (Libro no debería, además, imprimir facturas).
- ▶ Alta cohesión / bajo acoplamiento: los miembros de una clase deben estar fuertemente relacionados entre sí, y las clases deben depender lo mínimo posible unas de otras.
- ▶ Usar const en métodos y parámetros que no modifican el objeto o el argumento.
- ▶ Evitar variables globales; preferir encapsular el estado dentro de objetos.

Principios y su beneficio

Principio	Beneficio
Encapsulamiento	protege invariantes del objeto
Nombres descriptivos	código auto-explicativo
No duplicar código	un solo lugar para corregir bugs
Bajo acoplamiento	cambios localizados, menos efectos colaterales

? ¿Qué principio se viola si la clase Libro también contiene la lógica para generar el reporte PDF del catálogo completo?

Consolidado de errores frecuentes de todo el repaso

Bloom: Evaluar

 8 min

- ▶ Esta tabla resume, por tema, los errores más frecuentes.
- ▶ Revisala como checklist final antes del examen/tp.

Checklist de errores frecuentes

Tema	Error frecuente
Relaciones	confundir composición con herencia
Herencia	usarla sólo para reutilizar código, sin ES-UN
Memoria	olvidar delete / delete[] (memory leak)
Polimorfismo	olvidar 'virtual' en la clase base
Copia	shallow copy no intencional -> double delete
Recursividad	caso base ausente o mal definido
Cadenas C	usar == en vez de strcmp
Templates	no repetir 'template<typename T>' fuera de la clase
Operadores	operator<< como método en vez de friend

? De esta lista, ¿cuáles dos errores considerarás que son los más fáciles de cometer bajo presión de tiempo en el parcial?

MÓDULO

13

≈ 15 minutos

Ejercicio Integrador

Objetivos de esta sección

- Diseñar un fragmento del Sistema de Biblioteca integrando todos los temas del repaso.
- Justificar cada decisión de diseño (relación, uso de virtual, memoria, templates).

Diseño integrador: Sistema de Biblioteca

Consigna: diseñá en papel las clases necesarias para modelar Persona, Bibliotecario, Socio, Libro, LibroDigital y Prestamo, aplicando:

- ▶ Encapsulamiento en todas las clases (atributos privados, getters/setters necesarios).
- ▶ Asociación entre Prestamo y Socio / Libro.
- ▶ Agregación entre Biblioteca y Socio.
- ▶ Composición entre Prestamo y Comprobante.
- ▶ Herencia: Bibliotecario y Socio derivan de Persona; LibroDigital deriva de Libro.
- ▶ Polimorfismo: método virtual presentarse() o similar en Persona.
- ▶ Sobrecarga de operadores: operator<< para imprimir un Prestamo.
- ▶ Memoria dinámica: un vector dinámico para el catálogo.
- ▶ Templates: una Class Template Repositorio<T> reutilizable para libros y socios.

Boceto UML esperado

```
classDiagram
    class Persona
    class Bibliotecario
    class Socio
    class Libro
    class LibroDigital
    class Prestamo
    class Comprobante
    class Biblioteca
    class Repositorio_T["Repositorio<T> (template)"]

    Persona <|-- Bibliotecario
    Persona <|-- Socio
    Libro <|-- LibroDigital
    Prestamo *-- Comprobante
    Prestamo --> Socio
    Prestamo --> Libro
    Biblioteca o-- Socio
    Repositorio_T <|-- Biblioteca
    Repositorio_T <|-- Libro
```

Preguntas teóricas

¿Cuál es la diferencia esencial entre una clase y un objeto?

¿Qué diferencia hay entre encapsulamiento y abstracción?

¿Por qué la herencia debe usarse para modelar ES-UN y no sólo para reutilizar código?

¿Qué es el enlace dinámico y qué palabra clave lo habilita en C++?

¿Qué diferencia hay entre delete y delete[]? ¿Cuándo se usa cada uno?

¿Qué es un memory leak y cómo se produce típicamente?

¿Qué diferencia hay entre shallow copy y deep copy?

¿Cuándo un operador debe implementarse como friend en vez de como método?